Microcontroller Notes

Cheat Sheets with Design Guidelines

1. Von Neumann vs Harvard Architecture

1.1 Von Neumann Architecture

- Single memory for both instructions and data.
- Instructions and data share the same bus.
- Simple hardware design and lower implementation cost.
- Suffers from the Von Neumann bottleneck.

When to Use Von Neumann Architecture

- In low-cost embedded systems where hardware simplicity is critical.
- When system performance requirements are modest.
- In microcontroller-based designs with limited memory.
- Educational systems and basic control applications.
- When reduced power consumption and smaller silicon area are desired.

1.2 Harvard Architecture

- Separate memories for instructions and data.
- Separate buses enable parallel access.
- Higher performance and improved throughput.
- Increased hardware complexity and cost.

When to Use Harvard Architecture

- In high-performance and real-time embedded systems.
- When deterministic execution timing is important.
- Digital signal processing, multimedia, and communication systems.
- When instruction fetch and data access must occur simultaneously.
- Applications where performance is prioritized over cost.

2. RISC vs CISC Architecture

2.1 RISC (Reduced Instruction Set Computer)

- Simple and small instruction set.
- Fixed-length instructions.

- Load/store architecture.
- Large number of general-purpose registers.

When to Use RISC Architecture

- In low-power and battery-operated systems.
- When high clock speeds and efficient pipelining are required.
- In microcontroller and embedded processor designs.
- Applications with repetitive, arithmetic-heavy workloads.
- When simplicity and scalability of hardware are important.

2.2 CISC (Complex Instruction Set Computer)

- Large and complex instruction set.
- Variable-length instructions.
- Supports memory-to-memory operations.
- Smaller program size.

When to Use CISC Architecture

- When code size must be minimized.
- In systems performing complex data manipulation and string operations.
- When hardware can support complex instruction decoding.
- In legacy systems and backward-compatible architectures.
- Applications where fewer instructions per program are desirable.

3. Raspberry Pi vs Arduino

3.1 Raspberry Pi (Microprocessor-Based System)

- Microprocessor-based platform running a full operating system.
- High processing capability and memory resources.
- Supports multitasking, networking, and multimedia interfaces.

When to Use Raspberry Pi

- In applications requiring complex computation or decision-making.
- When image processing, video analysis, or AI algorithms are involved.
- Systems requiring internet connectivity or cloud integration.
- Prototyping of smart and IoT-based systems.
- When software flexibility is more important than strict real-time timing.

3.2 Arduino (Microcontroller-Based System)

- Microcontroller-based platform with no operating system.

- Deterministic execution and low power consumption.
- Limited memory and processing capability.

When to Use Arduino

- In low-cost embedded systems with simple control logic.
- When precise timing and hard real-time behavior are required.
- Battery-powered and energy-efficient applications.
- Direct interfacing with sensors and actuators.
- Systems where reliability and simplicity are critical.